

Project Proposal And Requirements Analysis

1. Team Information

Name	Student number	Main responsibilities
Lennart Axel Conrad	2025080264	Chess-domain architecture, C++ core implementation, neural-network/evaluator paths, tests, model validation, build/package work, documentation integration.
Erik Mkrtchyan	2025080273	raylib interaction workflows, visual layout review, activation presentation, manual UI testing, tutorial/demo preparation, user-facing documentation review.

Both team members should be able to explain the build process, major classes, source layout, runtime workflow, tests, and final demo behavior.

2. Project Topic

The project is a native C++ chess neural-network visualizer. It combines a complete legal human-vs-human chess board with visualization panels for:

- NNUE HalfKP accumulator evaluation,
- LC0-style CNN residual evaluation,
- LC0 BT4-style token-transformer evaluation,
- handcrafted Classical evaluation.

It also includes a native PUCT MCTS search-tree workbench. The topic is more advanced than a basic chess game because it includes legal move generation, notation, file I/O, binary model loading, tensor operations, search, tests, and real-time visualization.

3. Prior Work Used As Reference

The submitted runtime is C++. Prior personal chess projects were used as reference material only:

- <https://github.com/LenniAConrad/chess-rtk>: Java chess research toolkit with move generation, notation, evaluation, engine/search, datasets, and visualization concepts.
- <https://github.com/LenniAConrad/chess-web>: TypeScript chess player/trainer with browser UI and gameplay workflows.

The C++ application does not call either repository at runtime.

4. Course Requirement Mapping

Requirement	Project evidence
Classes and encapsulation	Position, Game, MoveHistory, Tensor, ActivationSnapshot, Nnue/Cnn/Bt4 Network, Mcts, App, BoardView, activation views, editor widgets.

Requirement	Project evidence
Multi-file C++ structure	Separate <code>.h/.cpp</code> files under <code>src/chess</code> , <code>src/game</code> , <code>src/io</code> , <code>src/nm</code> , <code>src/search</code> , and <code>src/viz</code> .
File I/O	<code>Config</code> load/save, FEN load/save, PGN export helpers, binary model readers, test reference data, Windows package config/assets/models.
Inheritance/polymorphism	<code>INetwork</code> implemented by NNUE/CNN/BT4 networks; <code>IActivationView</code> implemented by NNUE/CNN/BT4/Classical views.
Templates	<code>Tensor<T, Rank></code> generic tensor container.
Linked list	<code>MoveHistory</code> manually manages a doubly linked list of move records for undo/redo.
Arrays/containers	Bitboards, fixed move arrays, mailbox board, STL vectors/maps/unordered maps for histories, tensors, snapshots, model weights, and search nodes.
Chess game	Complete human-vs-human legal board with promotion, undo/redo, move history, FEN, and setup editor.
Advanced feature	NN/evaluator activation visualizer plus native PUCT MCTS tree workbench.
User manual	<code>docs/user-manual.md</code> .
Design specification	<code>docs/design-spec.md</code> and <code>docs/class-diagram.png</code> .
Test cases	<code>docs/test-cases.md</code> , <code>tests/*.cpp</code> , and screenshots under <code>docs/screenshots/</code> .
Summary report	<code>docs/summary-report.md</code> .
AI usage	<code>docs/ai-usage.md</code> .
Demo media	<code>docs/tutorial.gif</code> , <code>docs/demo.mp4</code> , and screenshot stills under <code>docs/screenshots/</code> .
Packaged build	Linux build scripts and Windows x64 package under <code>dist/</code> .

5. Functional Requirements

1. The user can play a legal chess game with mouse input.
2. The board shows selected pieces, legal destinations, last move highlights, promotion choices, and status text.
3. The user can undo, redo, reset, flip, and play a random legal move.
4. The user can load a complete FEN string and save the current FEN.
5. The user can create custom legal positions in setup mode and cannot apply invalid positions.
6. The user can switch between NNUE, CNN, BT4, Classical, and Off activation modes without changing the chess position.
7. The visualizer displays meaningful architecture-specific data for each evaluation path.
8. The user can run and inspect a live search preview from the current position.
9. The user can open the MCTS tree workbench, run PUCT search, pan/zoom the tree, filter it, scrub growth frames, and trace selected nodes.
10. The project can be built, tested, documented, and packaged from command line scripts.

6. Non-Functional Requirements

- The chess core must be independent of raylib.
- CPU execution must be available without CUDA.
- Runtime code must not depend on Java, TypeScript, Python, external chess engines, or online services.

- Compact runtime model files should keep the demo portable.
- The UI should remain responsive while model evaluation and search run.
- The documentation should match the current source, config, and model files.
- The test suite should be runnable from a clean command line build.

7. Acceptance Plan

Before submission or defense:

1. Run `./scripts/test.sh`.
2. Run `./build/tests/cnnv_tests` if an explicit individual-test count is needed.
3. Run `./scripts/demo_check.sh`.
4. Launch the app with `./scripts/run.sh`.
5. Demonstrate board play, undo/redo, FEN load/save, setup validation, architecture switching, search preview, and MCTS tree view.
6. Regenerate PDFs with `./scripts/export_docs.sh` when Markdown documents are changed.
7. Rebuild the Windows package when source, assets, config, or model defaults change.